

TECHNICAL DOCUMENTATION

BEM CRM Dashboard — Complete Codebase Reference

A full-stack Customer Relationship Management platform powering sales / CSR operations, technical delivery, and accounts — six role portals, real-time chat, multi-currency payments, and layered security, built on Next.js 16 + Express 5 + MongoDB.

6 Role Portals

~42 Dashboards

10 API Modules

WebSocket Chat

Repository: CRM/ (monorepo — backend/ + frontend/)

Prepared for: Engineer Mohsin Qayyum · Managing Partner

Version: August 2026 · includes Messages, Notifications, Presence & full dashboard merges

§ Document At a Glance

This reference consolidates the entire BEM CRM codebase — architecture, database, backend, API surface, frontend, security, business logic, real-time systems, and every dashboard's data sources and merges — into a single visual document. Nothing from the source has been dropped; diagrams and charts have been added throughout for clarity.

6

ROLE PORTALS

~42

DASHBOARD PAGES

10

API ROUTE MODULES

4

BACKGROUND CRON JOBS

12

PAYMENT BUCKETS (4×3)

200MB

MAX CHAT FILE SIZE

Dashboard Pages per Portal (~42 total)

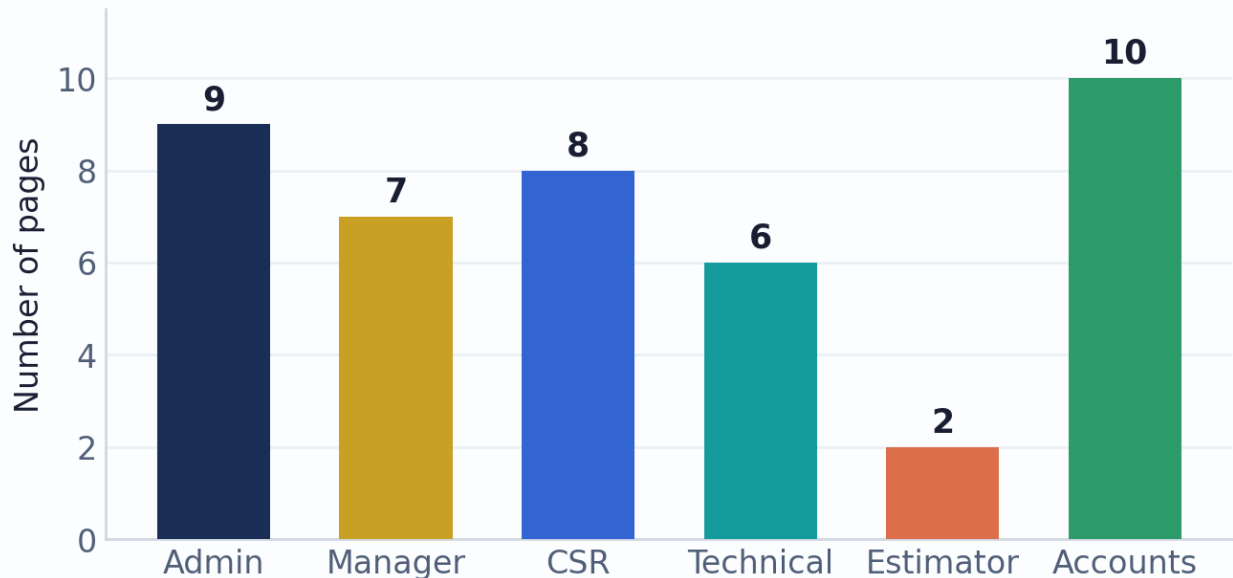


Figure 1 — Dashboard pages distributed across the six role portals (Admin, Manager, CSR, Technical, Estimator, Accounts).

Technology Stack Backend



Express 5
Prisma/Mongo
JWT+bcrypt
ws (WebSocket)
multer/xlsx
Nodemailer

Frontend



Next.js 16
React 19
Tailwind v4
Axios
Recharts
next-themes

Figure 2 — Technology stack: backend (Express/Prisma/Mongo) and frontend (Next.js/React/Tailwind).

§ Table of Contents

01	Overview	13	Chat / Messages System
02	Architecture	14	Notifications & Alerts
03	Tech Stack	15	Presence, Heartbeat & Work Sessions
04	Repository Structure	16	Background Jobs (Cron)
05	Database (MongoDB + Prisma)	17	Caching & Performance
06	Backend	18	External Services
07	API Reference	19	Environment Variables
08	Frontend	20	Local Development
09	Authentication & Sessions	21	Deployment Notes
10	Security	22	Every Dashboard — Sources & Merges
11	Lead Workflow & Business Logic	23	Quick Reference
12	Uploads & Data Import		

01 Overview

This is a **full-stack Customer Relationship Management (CRM)** system built for **BEM Solutions**. It powers daily sales / CSR operations plus technical delivery and accounts:

- **Lead assignment** to CSRs with status workflows (pending, not picked, not interested, important, interested, etc.)
- **Potential clients** pipeline (important / scheduled / interested)
- **Client directory** (new vs returning / old clients)
- **Active projects / Project DB** (won deals with phases, deadlines, multi-currency payments)
- **Payments dashboard** (Admin/Manager) — collections by source (AH / SN / CHQ / HM), USD·CAD·PKR, full payment history
- **Technical portal** — assignments, deadlines, KPI, reports (Technical Manager / Estimators)
- **Accounts portal** — income statement, team salaries / payroll (PF, occasional), balance sheet, cash flow
- **Admin bulk Excel uploads** with CSR round-robin distribution
- **Internal real-time chat / Messages** (WebSocket + REST fallback; org pools + ACL)
- **In-app notifications** — Admin/Manager meetings bell, CSR schedule + stats, TM project inbox, concurrent-login alerts
- **Work sessions** — CSR/Estimator check-in, dashboard vs away time, Admin session report
- **Reports**, audit logs, access history, and layered security controls

User Roles

Role	Route prefix	Purpose
admin	/admin/*	Full control: users, settings, stealth CSR chat, shuffle, CSR purge, access history, payments
manager	/manager/*	Leads, reports, projects, uploads, bin, payments — reuses admin pages (no Users / Settings)
csr	/csr/*	Own assigned leads, status updates, clients, projects, bin, chat
technical_manager	/technical/*	Technical dashboard, active projects, assignments, KPI, reports
estimator	/estimator/*	Assigned takeoff work, notes, time tracking
accounts	/accounts/*	Income statement, payroll, balance sheet, cash flow, assets

Login requires selecting the correct **portal**. Backend enforces that `loginRole` must match the account role.

Seed Accounts (backend/prisma/seed.ts)

Email	Password	Role	Notes
admin@gmail.com	123456	admin	—
manager@gmail.com	123456	manager	—
csr@gmail.com	123456	csr	csrCode: CSR001

SECURITY NOTE

These are development seed credentials. They must be rotated / disabled before any production exposure.

02 Architecture

The platform is a monorepo: a Next.js 16 App-Router frontend that both renders the UI and acts as a reverse proxy, in front of an Express 5 API which owns all business logic, the WebSocket chat server, and the scheduled cron jobs.

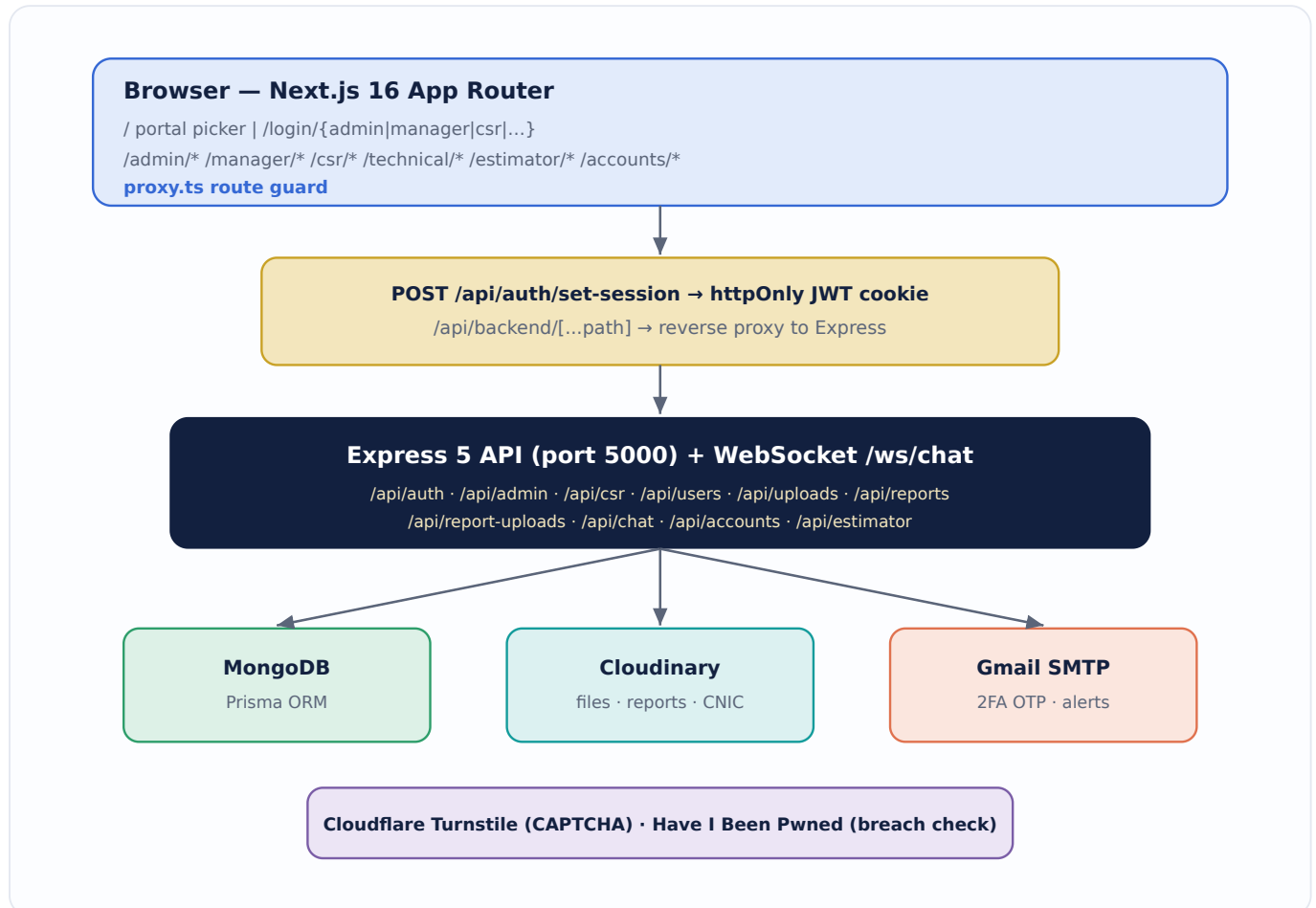


Figure 3 — End-to-end system architecture: browser → Next.js proxy → Express API → MongoDB / Cloudinary / Gmail SMTP, gated by Turnstile & HIBP.

Request Flow (Development)

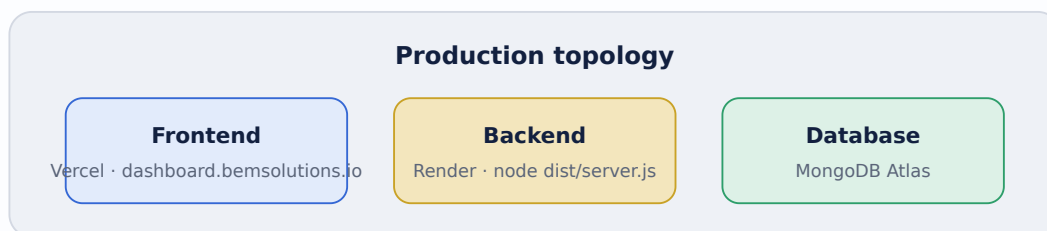


Figure 4 — A CSR stats request travels through the same-origin proxy, credential attachment, token verification, and role gate before hitting the cached handler.

Production Topology

Component	Host	Detail
Frontend	Vercel	dashboard.bemsolutions.io
Backend	Render / Node host	node dist/server.js
Database	MongoDB Atlas	or compatible

03 Tech Stack

Backend (backend/)

Layer	Technology
Runtime	Node.js, TypeScript 5.8
Framework	Express 5.1
ORM	Prisma 6.19 → MongoDB
Auth	JWT + httpOnly cookies, bcryptjs
Security	helmet, cors, express-rate-limit, login logout
Realtime	<code>ws</code> WebSocket (<code>/ws/chat</code>)
Files	multer (memory), xlsx, Cloudinary
Email	Nodemailer (Gmail SMTP)
Dev	ts-node-dev
Prod start	<code>node dist/server.js</code> (committed dist/)

Frontend (frontend/)

Layer	Technology
Framework	Next.js 16.2 (App Router), React 19
Styling	Tailwind CSS v4, CSS variables (light/dark)
HTTP	Axios (<code>lib/api.ts</code>)
Realtime	WebSocket client (<code>lib/chatSocket.ts</code>)
Charts	Recharts (lazy-loaded on Reports)
Themes	next-themes
Icons	lucide-react, react-icons
PDF	jspdf + jspdf-autotable
Toasts	react-hot-toast

04 Repository Structure

A single repository holds both apps. The backend ships a committed `dist/` so production can start without a build step, and `proxy.ts` holds the route-guard logic on the frontend.

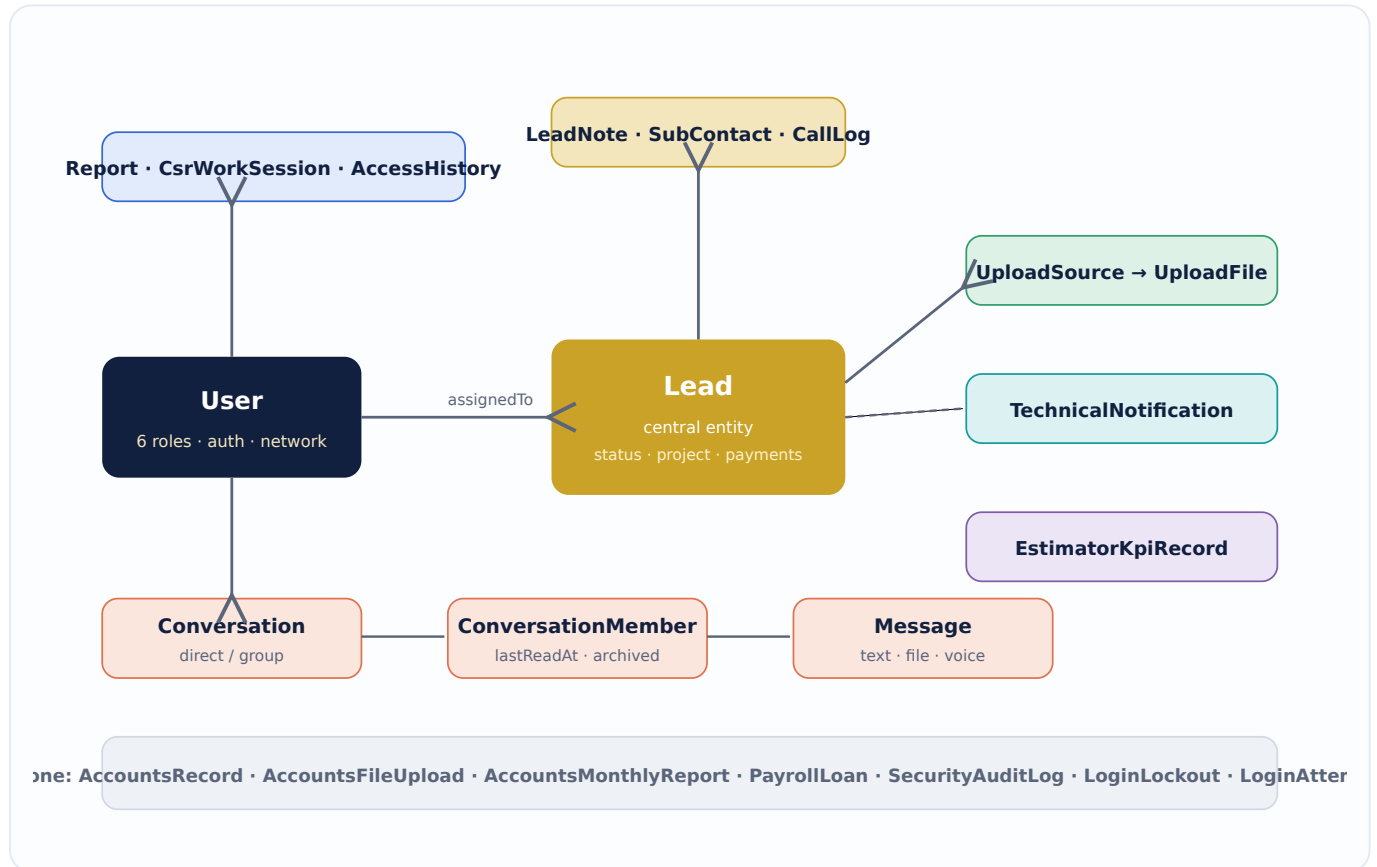
```
CRM/
├── CODEBASE.md
├── backend/
│   ├── prisma/
│   │   ├── schema.prisma
│   │   └── seed.ts
│   ├── dist/                # Compiled JS (used by npm start)
│   └── src/
│       ├── server.ts        # HTTP + WS + cron jobs
│       ├── app.ts           # Express middleware + route mount
│       ├── chat/            # chatSocket.ts, chatMessageFormat.ts
│       ├── config/          # db, env, cloudinary, security
│       ├── controllers/     # auth, user, upload, report (5 files)
│       ├── middleware/      # auth, role, rateLimit, upload, adminIp
│       ├── routes/          # 8 route modules (heavy inline logic)
│       └── utils/           # ~30 domain helpers
├── frontend/
│   ├── proxy.ts             # Route guard (+ config.matcher)
│   └── app/
│       ├── page.tsx         # Portal picker (Admin / Manager / CSR)
│       ├── login/[portal]/  # Per-portal login
│       ├── layout.tsx       # Root: fonts, theme, toast
│       ├── api/             # auth (set/clear-session) + backend proxy
│       ├── admin/ manager/ csr/ technical/ estimator/ accounts/
│       ├── components/      # Shared: chat, auth, projects, staff, tables
│       └── lib/             # api, session, caches, paymentsSummary, projectFields
```

WIRING NOTE

`proxy.ts` exports `proxy()` and `config.matcher`. A root `middleware.ts` must re-export it if the Next.js version requires that wiring for route protection to run.

05 Database (MongoDB + Prisma)

Provider: MongoDB (`DATABASE_URL`) · **IDs:** `cuid()` mapped to `_id` via `@map("_id")` · **Lead statuses:** plain strings (not Prisma enums) for workflow flexibility.



User (auth + HR + network + chat ACL)

Field	Notes
<code>role</code>	admin csr manager technical_manager estimator accounts
<code>csrCode</code> / <code>employeeCode</code>	Client coding vs HR employee ID; used in <code>clientCode</code> generation
<code>cnic</code> , <code>cnicPdfUrl</code>	Identity docs (Cloudinary; CNIC PDF as image/pdf for preview)
<code>profilePic</code>	Cloudinary URL
<code>isActive</code>	<code>false</code> after 90-day CSR inactivity (cron)
<code>isOnline</code> , <code>lastActive</code>	Presence via heartbeat
<code>twoFAEnabled</code>	Email OTP on login
<code>tokenVersion</code>	Incremented on password change / logout-all
<code>activeSessionId</code>	Single active session binding (<code>sid</code> in JWT)
<code>allowedIps[]</code>	Max 5 in-house approved IPs
<code>allowedMacAddress</code>	In-house device ID (via <code>X-Client-Mac</code> header)
<code>temporaryAccessIp/Mac/Until</code>	Temporary outside access (9-10 hours, auto-expires)
<code>fatherName</code> , <code>currentAddress</code> , <code>contactNo</code>	HR / profile fields
<code>chatEnabled</code>	Master switch — <code>false</code> = no Messages UI / WS / send
<code>chatAllowedUserIds</code>	Empty = message anyone allowed by org; else allowlist only
<code>chatVisibleToUserIds</code>	Empty = visible in directories; else only those viewers see this user

Indexes: `role` , `isActive` , `lastActive` , `[role,isActive]` , `temporaryAccessUntil`

Lead (central entity)

Field	Notes
<code>status</code>	Default <code>"pending"</code> — see Lead Statuses (§8)
<code>assignedTo</code>	FK → User (CSR)
<code>important</code> , <code>interested</code>	Boolean flags synced with status
<code>nextSchedule</code> , <code>nextTime</code> , <code>timezone</code>	Scheduled important calls
<code>followUpNotes</code>	Multi-line try history (NI / NP)
<code>interestedService</code>	Service when interested
<code>projectTitle</code> , <code>projectCode</code> , <code>projectDeadline</code> ...	Active project fields
<code>projectPhase</code>	<code>not_started</code> , takeoff/pricing/QA phases
<code>projectPayments</code>	JSON string of payment entries (see §22)
<code>technicalReceivedAt</code> , <code>pricingReceivedAt</code>	Handoff timestamps into Technical / Pricing
<code>technicalAssignments</code> , <code>technicalNotes</code>	JSON: estimator assignments + working notes
<code>projectWorkStatus</code> , <code>hiddenOnTechnical</code> , <code>needsRevision</code>	Technical workflow flags
<code>revisionRequestedAt</code> , <code>revisionNotes</code>	Admin → TM revision queue
<code>deadlineTiming</code>	<code>morning</code> <code>cob</code> emergency submit window (TM notified on change)
<code>clientCode</code>	Auto on Close Client: <code>{csrCode}-01</code> , etc.
<code>isOldClient</code>	Auto if company had prior won project
<code>notInterestedCount</code> , <code>notPickedCount</code>	Retry counters (limits: 3 / 7)
<code>inBin</code> , <code>binReason</code> , <code>binPreviousStatus</code>	Bin / soft-delete
<code>hiddenOnMain</code>	Hides from CSR main pipeline
<code>uploadedFileId</code>	FK → UploadFile
<code>shuffleAttempts</code>	Lead shuffle cron counter

Indexes: `assignedTo` , `status` , `phone` , `email` , `company` , `createdAt` , `updatedAt` , `important` , `interested` , `inBin` , `nextSchedule` , `hiddenOnMain` + composites for tab/bin queries.

Other Models

Model	Purpose
<code>LeadNote</code>	Timestamped notes per lead — edit bumps <code>createdAt</code> so note sorts to top as latest
<code>SubContact</code>	Extra contacts on a lead
<code>CallLog</code>	CSR status-change history (feeds CSR 24h notification stats)
<code>UploadSource</code>	Admin source label + <code>sudoName</code> (shown to CSRs)
<code>Trade</code>	Trade labels for uploads / projects — <code>/admin/trades</code> CRUD
<code>UploadFile</code>	Excel upload metadata + Cloudinary URL
<code>Report</code>	Generated or uploaded reports (cron JSON + manual report-uploads)
<code>Conversation</code>	Chat thread (direct / group)
<code>ConversationMember</code>	Membership, <code>lastReadAt</code> , <code>archivedAt</code>
<code>Message</code>	Text/file/voice; reactions JSON; replies; mentions
<code>SecurityAuditLog</code>	Auth & security events
<code>AccessHistory</code>	Login/logout/heartbeat per user
<code>LoginLockout</code>	Failed login tracking
<code>LoginAttemptAlert</code>	Concurrent login attempt — drives session notifier + WS push
<code>RateLimitEntry</code>	Optional Prisma rate-limit store
<code>CsrWorkSession</code>	CSR and Estimator check-in sessions (<code>dashboardMs</code> / <code>awayMs</code> , device, IP, <code>closedReason</code>)
<code>PayrollLoan</code>	Accounts loan ledger (employee + department + paid/total)

AccountsRecord	Day JSON keyed by AccountsPage enum + recordDate
AccountsFileUpload	Proof / attachment uploads per accounts page/date
AccountsMonthlyReport	Saved Word monthly archives
EstimatorKpiRecord	Monthly KPI marks/stars/remarks per estimator
TechnicalNotification	TM inbox rows — Admin/Estimator/system producers

ACCOUNTSPAGE ENUM

dashboard , balance_sheet , income_statement , cash_flow , total_assets , sales_payroll , technical_payroll , income_budget .

06 Backend

Entry & App bootstrap

`src/server.ts` loads `.env` + `validateEnv()`, creates the HTTP server, attaches Express, initializes WebSocket chat (`initChatSocket` on `/ws/chat`), and runs the cron jobs (§16). `src/app.ts` mounts middleware then routes:

Middleware	Purpose
<code>helmet</code>	CSP (prod), HSTS, security headers
<code>cors</code>	localhost, bemsolutions.io, Vercel, <code>FRONTEND_URL</code>
<code>express.json</code>	1 MB limit
<code>cookieParser</code>	JWT cookie
<code>apiLimiter</code>	300 req/min on <code>/api</code> (in-memory by default)

Route Mounting

Prefix	File	Notes
<code>/api/auth</code>	<code>auth.routes.ts</code>	Login, 2FA, profile
<code>/api/admin</code>	<code>admin.routes.ts</code>	<code>enforceAdminIpAllowlist</code> on all routes
<code>/api/csr</code>	<code>csr.routes.ts</code>	~1,200 lines — most CSR business logic
<code>/api/users</code>	<code>user.routes.ts</code>	User CRUD + network policy
<code>/api/uploads</code>	<code>upload.routes.ts</code>	Excel import
<code>/api/reports</code>	<code>report.routes.ts</code>	Performance reports
<code>/api/report-uploads</code>	<code>reportUpload.routes.ts</code>	Manual report files
<code>/api/chat</code>	<code>chat.routes.ts</code>	Chat REST + admin stealth view
<code>/api/accounts</code>	<code>accounts.routes.ts</code>	Accounts dashboard data / uploads
<code>/api/estimator</code>	<code>estimator.routes.ts</code>	Estimator project notes & assignments

Controllers

File	Responsibility
<code>auth.controller</code>	Login, 2FA, register, profile, password, audit, network policy
<code>user.controller</code>	User CRUD, network fields, admin password reset
<code>upload.controller</code>	Excel parse, chunked insert (1000), batched update (30), CSR distribution
<code>report.controller</code>	Period reports + internal 90s/300s cache
<code>reportUpload.controller</code>	Report file upload/delete

Most admin/CSR lead logic lives **inline in route files**, not controllers.

Middleware

File	Purpose
<code>auth.middleware</code>	JWT verify, session binding, network access, <code>lastActive</code> touch
<code>role.middleware</code>	<code>allowRoles(...)</code>
<code>rateLimit.middleware</code>	Login 15/15min; API 300/min; Prisma store only if flag set
<code>adminIp.middleware</code>	<code>ADMIN_ALLOWED_IPS</code> gate on <code>/api/admin/*</code>
<code>upload.middleware</code>	Multer: images 5MB, Excel 15MB, PDF 10MB

Key Utils (~30 domain helpers)

File	Purpose
<code>cache.ts</code>	In-memory TTL cache, pattern invalidation, throttled heal lock
<code>binLeads.ts</code>	Bin rules (NI=3, NP=7), <code>buildBinWhere</code> , pipeline where-clauses
<code>csrDashboardQueries.ts</code>	CSR tab stats, paginated tab leads, potential leads
<code>csrLeadSelect.ts</code>	Lean Prisma <code>select</code> shapes (max 5 notes on client list)
<code>adminClientQueries.ts</code>	Admin list filters (important/schedule/interested/closed) + search / state / phone / areaCodes
<code>adminSummaryStats.ts</code>	Admin dashboard tab counts + chart series
<code>potentialClientBuckets.ts</code>	Important / interested / scheduled bucket rules

<code>leadClientPromotion.ts</code>	Won-client promotion, pending dedupe, <code>healPipelineLeaksForCsr</code>
<code>leadPhoneLookup.ts</code> · <code>leadDuplicateCheck.ts</code>	Returning client + duplicate detection
<code>projectLeads.ts</code> · <code>projectPayments.ts</code>	Active/won filters; payment JSON parse/totals (AH/SN/CHQ/HM · USD/CAD/PKR)
<code>paymentsSummary.ts</code>	Admin/Manager Payments dashboard payload
<code>projectCurrencyCollections.ts</code>	Bridge: project payments → Accounts currency month buckets
<code>technicalEstimators.ts</code> · <code>technicalNotifications.ts</code>	Assignments/notes JSON; TM inbox rows + field-diff
<code>technicalMonitoring.ts</code> · <code>technicalKpiRecords.ts</code>	Workload/timer/overdue; KPI CRUD + previous-month ratings
<code>chatAccess.ts</code>	Org pools (GPS vs BEM), allowlist/visibility, directory filter
<code>csrSessionTracking.ts</code>	Work-session check-in/out/heartbeat (CSR + Estimator); 16h auto-close
<code>fxRates.ts</code> · <code>accountsStorage.ts</code>	Live FX; Accounts day-record persistence
<code>businessCalendar.ts</code> · <code>estTime.ts</code>	Business day / EST helpers
<code>securityEmail.ts</code> · <code>networkAccess.ts</code>	Login alert emails; per-user IP/MAC + temporary access
<code>uploadExcelParse.ts</code> · <code>uploadCategory.ts</code>	xlsx row parsing + classification; category enum
<code>deleteLeadsCascade.ts</code> · <code>purgeCsrLeads.ts</code>	Cascade delete + cache invalidation; bulk purge
<code>sessions.ts</code> · <code>loginLockout.ts</code>	OTP store, <code>activeSessionId</code> , alerts; 8 fails → 30min, CAPTCHA after 3
<code>auditLog.ts</code> · <code>accessHistory.ts</code> · <code>rateLimitStore.ts</code>	Audit writes; login/heartbeat history (throttled); Prisma rate-limit store

07 API Reference

All endpoints are prefixed with `/api` and protected by `verifyToken` unless noted. Below is the complete surface across all 10 route modules.

Auth — `/api/auth`

Method	Path	Auth	Description
GET	/captcha-config	Public	Turnstile site key
POST	/login	Public	Login (+ portal role, network policy, CAPTCHA)
POST	/verify-otp	Public	2FA OTP
POST	/resend-otp	Public	Resend OTP
POST	/register	Admin	Create user
POST	/logout	Auth	Logout
GET	/me	Auth	Current user
GET	/session-status	Auth	Login-attempt alert status
POST	/session-alert/ack	Auth	Dismiss alert
POST	/logout-all	Admin/Mgr	Invalidate all sessions
GET	/audit-logs	Admin	Security audit log
PUT	/profile	Auth	Update profile
PUT	/change-password	Auth	Change password
PUT	/toggle-2fa	Auth	Enable/disable 2FA

Admin — `/api/admin` (`verifyToken` + `role` + `enforceAdminIpAllowlist`)

Method	Path	Roles	Description
GET	/dashboard	admin	Legacy stub
GET	/notifications	admin, mgr, TM	Meetings (scheduledCalls) + technical notes (§14)
GET	/csrs	admin, mgr	Active CSR list
GET	/clients/tab-counts	admin, mgr, TM	Tab badge counts — search/state/phone/areaCodes/scheduleDate
GET	/clients/summary	admin, mgr	Dashboard stats + charts
GET	/clients	admin, mgr, TM	Paginated leads (view/search/state/phone/areaCodes/scheduleDate/csr)
GET/PUT/DEL	/lead/:id	admin, mgr	Lead CRUD
PUT	/lead/:id/reassign	admin, mgr	Reassign to CSR
POST/PUT/DEL	/lead/:id/notes[:notelId]	admin, mgr, TM	Notes CRUD — PUT bumps createdAt
POST/PUT/DEL	/lead/:id/subcontacts[:subId]	admin, mgr	Sub-contacts
GET	/projects	admin, mgr	Paginated active projects
GET	/projects/excel-template	admin, mgr	Download Project DB Excel template
POST	/projects/excel-upload	admin, mgr	Bulk upload projects from Excel (any dates)
POST/PUT	/project, /project/:id	admin, mgr	Project CRUD
GET	/lookup-by-phone	admin, mgr	Phone lookup
GET	/bin	admin, mgr	Bin leads (buildBinWhere)
PUT	/csr-status/:id	admin	Activate/deactivate CSR
GET	/csr-history/:csrId	admin	CSR history
DEL	/csr-leads/:csrId	admin	Purge all CSR leads
GET/POST	/shuffle-status · /run-shuffle	admin	Shuffle preview / manual shuffle
*	/upload-sources[:id]	admin, mgr	Upload sources CRUD
GET	/access-history/:userId	admin	User access history

POST	/migrate-legacy-notes	admin	One-time migration
*	/trades[:id]	admin, mgr	Trade labels CRUD
GET	/payments-summary	admin, mgr	Payments dashboard (month)
PUT	/project/:id/send-final-submission	admin, mgr, TM	→ qa_phase
PUT	/project/:id/return-pricing	admin, mgr, TM	Final → pricing
PUT	/project/:id/hide-technical	admin, mgr, TM	Soft-hide from TM only
PUT	/project/:id/mark-completed	admin, mgr	Completed + hide TM + Completed tab
PUT	/project/:id/request-revision	admin, mgr	TM revision queue + inbox notify
PUT	/lead/:id/technical-notes/read	admin, mgr, TM	Stub success (UI dismiss companion)
GET	/technical-notifications	TM, admin	TM inbox list (?limit=&unread=)
PUT	/technical-notifications/read-all	TM, admin	Mark all read
PUT/DEL	/technical-notifications/:id[/read]	TM, admin	Mark one read / hard dismiss
GET	/technical-estimators	TM, admin, mgr	Estimator directory for assign
GET	/technical-monitoring	TM, admin	Live workload / timers
*	/technical-kpi*	TM, admin	KPI records + auto-fill
GET	/technical-previous-month-ratings	TM, admin, est*	Previous month stars/marks
GET	/csr-session-report	admin, mgr	Work-session hours report
PUT	/csr-work-session/:id	admin	Edit session times
GET/DEL	/csr-reset-pages · /csr-leads/:id/reset	admin	CSR page / targeted lead reset

* Estimator uses own </api/estimator/previous-month-rating> for self.

CSR — </api/csr>

Method	Path	Description
GET	/stats	Tab counts (cached; throttled pipeline heal)
GET	/tab-leads	Paginated leads by tab (today, pending, important...)
GET	/potential-leads	Important / schedule / interested bucket
GET	/dashboard	Legacy full dashboard payload (cached)
GET	/clients	Won clients (new vs old; cache-first, lean notes)
GET	/leads · /not-interested · /bin · /projects	Directory · NI/NP retry · bin · active projects
GET	/notifications	Scheduled calls + 24h CallLog stats (\$14)
GET	/activity-report	Activity by date range
GET/POST/PUT	/lead, /lead/:id	Lead CRUD + status workflow
PUT	/lead/:id/assign	CSR-to-CSR assign
GET	/lookup-by-phone · /csrs	Returning lookup · peer CSR list
POST	/project	Create project lead
GET/POST	/work-session · /check-in · /check-out	Current session · start/end work session
POST	/heartbeat · /offline	Presence + work-session; mark offline (beacon)
POST/PUT/DEL	/lead/:id/notes · /subcontacts	Notes & sub-contacts — PUT bumps createdAt

Uploads — </api/uploads>

M	Path	Roles
POST	/	admin, mgr
POST	/preview	admin, mgr
GET	/latest · /history	admin, mgr
PATCH	/:id/source	admin, mgr

Users — </api/users>

M	Path	Note
GET	/	CNIC masked for non-admin
POST	/upload-pic	Profile picture
POST	/upload-cnic	CNIC PDF (image/pdf, iframe-friendly)
PUT	/:id	Update + network + chat ACL

DEL	/:id	admin, mgr
-----	------	------------

PUT	/:id/password	Reset password
DEL	/:id	Delete + chat cascade

Reports

M	Path	Roles
GET	/reports/	admin, mgr
POST	/report-uploads/	admin
DEL	/report-uploads/:id	admin

Chat — [/api/chat](#)

Method	Path	Description
GET/POST	/conversations	List (?archived=1) filtered by ACL · create direct/group
DEL	/conversations/:id	Delete / leave
GET/POST	/conversations/:id/messages	Fetch / send (files up to 200 MB)
PATCH/DEL	/messages/:id	Edit (10 min) / delete
POST	/messages/:id/reactions	Toggle reaction
POST	/conversations/:id/read · /pin · /archive · /unarchive	Read · pin · per-user archive
POST/GET	/conversations/:id/typing	Typing (in-memory + WS)
POST	/status	Online/offline heartbeat (chat presence)
GET	/users · /conversations/:id/members	Directory (org pools + visibility) · group members
GET	/admin/viewable-users	Admin stealth: viewable users
GET	/admin/view/:userId/conversations · .../messages	Admin stealth read-only

Accounts — [/api/accounts](#)

M	Path
GET	/dashboard/summary · /fx-rates
GET	/income-budget/actuals
GET	/reports/monthly[/saved]
POST	/reports/monthly/save
*	/payroll-loans[/:id]
GET	/:page
GET/PUT	/:page/:date
POST	/:page/:date/upload

Estimator — [/api/estimator](#)

M	Path
GET	/dashboard · /projects · /lead/:id
PUT	/project/:id/timer · /complete · /takeoff
POST	/lead/:id/notes (→ TM notify)
GET	/previous-month-rating · /work-session
POST	/check-in · /check-out · /heartbeat · /offline

08 Frontend

Routes

Path	Page	Role
/	Portal picker	Public
/login/{admin manager csr}...	Login + 2FA	Public
/admin	Dashboard	Admin
/admin/users	User mgmt + IP/MAC policy + CNIC preview	Admin only
/admin/leads	All leads — tab counts follow filters	Admin
/admin/reports · /active-projects · /payments	CSR performance · Project DB · Collections	Admin
/admin/uploads · /bin · /settings	Excel uploads · Bin · Profile/2FA/audit	Admin
/manager ... /manager/payments	Same admin pages (no Users/Settings)	Manager
/csr	Call Data main dashboard	CSR
/csr/potential-clients · /not-interested · /clients	Buckets · NI/NP · directory	CSR
/csr/active-projects · /bin · /chat · /settings	Projects · Bin · Chat · 2FA	CSR
/technical ...	Dashboard / projects / reports	TM
/estimator ...	My-projects	Estimator
/accounts ...	Income / payroll / balance / cash flow / assets	Accounts

Layouts & init

Layout	Initializes
app/layout	Fonts, ThemeProvider, toast
admin / manager	StaffPresencelnit, StaffChatNotifier, SessionNotifierInit, ChatLayoutCleanup
csr	Above + DevToolsGuard, CsrPresencelnit
technical / estimator / accounts	Presence + chat notifier + session notifier + portal sidebar

Route protection — proxy.ts

- Parses JWT from httpOnly `token` cookie (role, exp, sid)
- Protects all six portals
- Redirects unauthenticated → `/`
- Redirects authenticated at `/` or `/login/*` → role home
- Cross-role redirects (admin may also enter `/accounts`)

API client — lib/api.ts

- Axios, `withCredentials:true` , 30s timeout
- Dev base: `/api/backend` ; Prod: `NEXT_PUBLIC_API_URL`
- Sends `Authorization: Bearer` + `X-Client-Mac`
- 401 on /admin or /csr → clear session → redirect /

CSR data architecture

Hook / API	Source
useCsrStats	/csr/stats via csrTabApiCache
useCsrTabLeads	/csr/tab-leads (paginated)
usePotentialLeads	/csr/potential-leads
useLeadsData / useProjectsData	/csr/clients · /csr/projects
useBinData / useNotInterested	/csr/bin · /csr/not-interested

Key UI Patterns

- **CSR tables:** `csrTableStyles.tsx` shared column layout; **Column filters:** `columnFilters.tsx`
- **Notes UI:** `CommentCell.tsx` — add/edit/delete; edited notes re-sort to top (timestamp bump)
- **Project editor:** `ProjectWorkbenchModal.tsx` — admin + CSR + technical (payment channel / currency / accounts month)
- **Payment detail cell:** `ProjectTableCells.tsx` — latest payment + **+N** expandable history
- **Payments dashboard:** KPIs, charts, multi-currency source cards, expandable project tables
- **Accounts payroll:** `payrollFormula.ts` , PF checkbox + occasional amount on all Team Salaries heads

09 Authentication & Sessions

Login is portal-scoped and layered: lockout & CAPTCHA, per-user network policy, optional breach check, optional 2FA, then a bound single-session JWT.

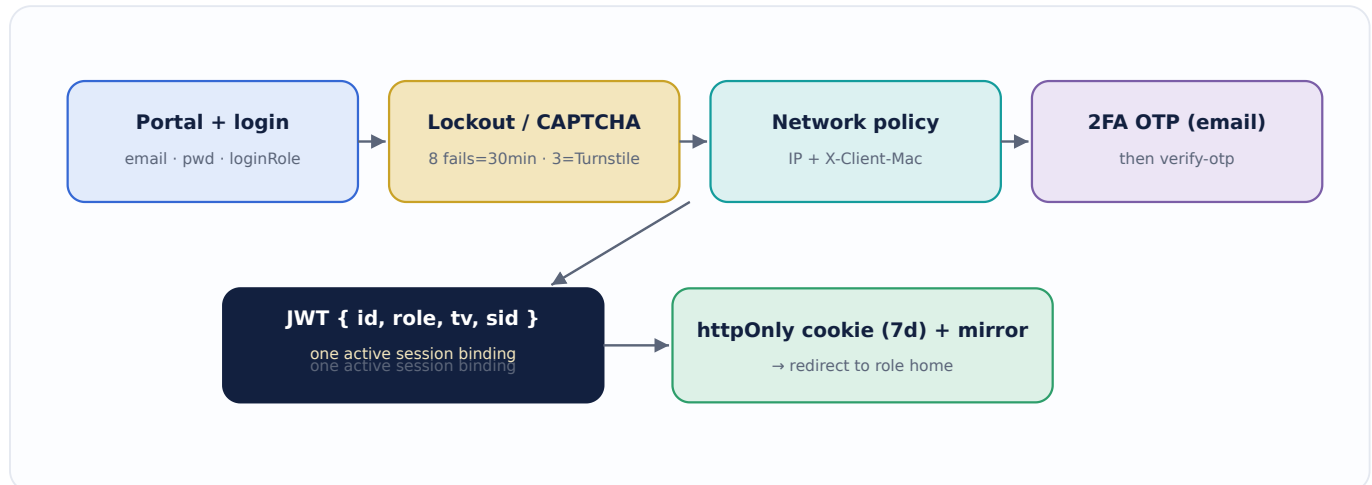


Figure 6 — Login pipeline: portal + credentials → lockout/CAPTCHA → network policy → optional 2FA OTP → bound JWT issued as an httpOnly cookie.

Login Flow

1. User picks portal → `/login/{admin|manager|csr}`
2. `POST /api/auth/login` with email, password, `loginRole`, optional Turnstile
3. Lockout check → CAPTCHA if needed → bcrypt verify
4. **Network access policy** evaluated (IP + `X-Client-Mac`)
5. Optional HIBP breach check
6. If 2FA → OTP email → `POST /api/auth/verify-otp`
7. JWT: `{ id, role, tv, sid }`
8. `setFrontendSession(token)` → httpOnly cookie (7 days) + sessionStorage
9. `POST /chat/status { online: true }` → redirect to dashboard

Session Binding

- **One active session** per user (`activeSessionId` must match JWT `sid`)
- `tokenVersion` incremented on password change / logout-all
- `verifyToken` rejects: missing `sid`, version mismatch, deactivated account, failed network policy

Per-User Network Access

- **In-house:** up to 5 `allowedIps` + one `allowedMacAddress`
- **Temporary outside:** `temporaryAccessIp/Mac` with `temporaryAccessUntil` (9 or 10 hours)
- Frontend sends persistent device ID as `X-Client-Mac`
- Enforced on login, OTP verify, and every authenticated request

10 Security

Backend controls

Control	Implementation
Password hashing	bcryptjs (10 rounds)
JWT secret	Min 32 chars in production
Rate limiting	Login 15/15min; API 300/min
Login lockout	8 fails / 30min (Prisma LoginLockout)
CAPTCHA	Turnstile after 3 fails
Password breach	Have I Been Pwned k-anonymity
Admin IP allowlist	ADMIN_ALLOWED_IPS on login + /api/admin/*
Per-user network policy	IP/MAC allowlist + temporary access
Portal role gate	Login role must match account role
Helmet + CORS	CSP (prod), HSTS, origin allowlist
Audit + access logs	SecurityAuditLog + AccessHistory (IP/UA/device)
CSR auto-deactivate	90 days without lastActive
File upload limits	Images 5MB · Excel 15MB · PDF 10MB
CNIC masking	Non-admins see masked CNIC

Frontend controls

Control	Implementation
Route guard	proxy.ts
DevTools guard	DevToolsGuard.tsx on CSR layout
Session invalidation	Axios 401 interceptor
Device identity	localStorage device ID → MAC header

DEFENSE IN DEPTH

Authentication combines credential, lockout, CAPTCHA, network policy, breach check, 2FA, and a single-session bound JWT — each layer is enforced on every authenticated request, not only at login.

11 Lead Workflow & Business Logic

The lead is the beating heart of the CRM. From a default `pending` state it branches into retry, interest, and scheduling paths — with automatic bin thresholds — and, when won, promotes into a full multi-currency project.

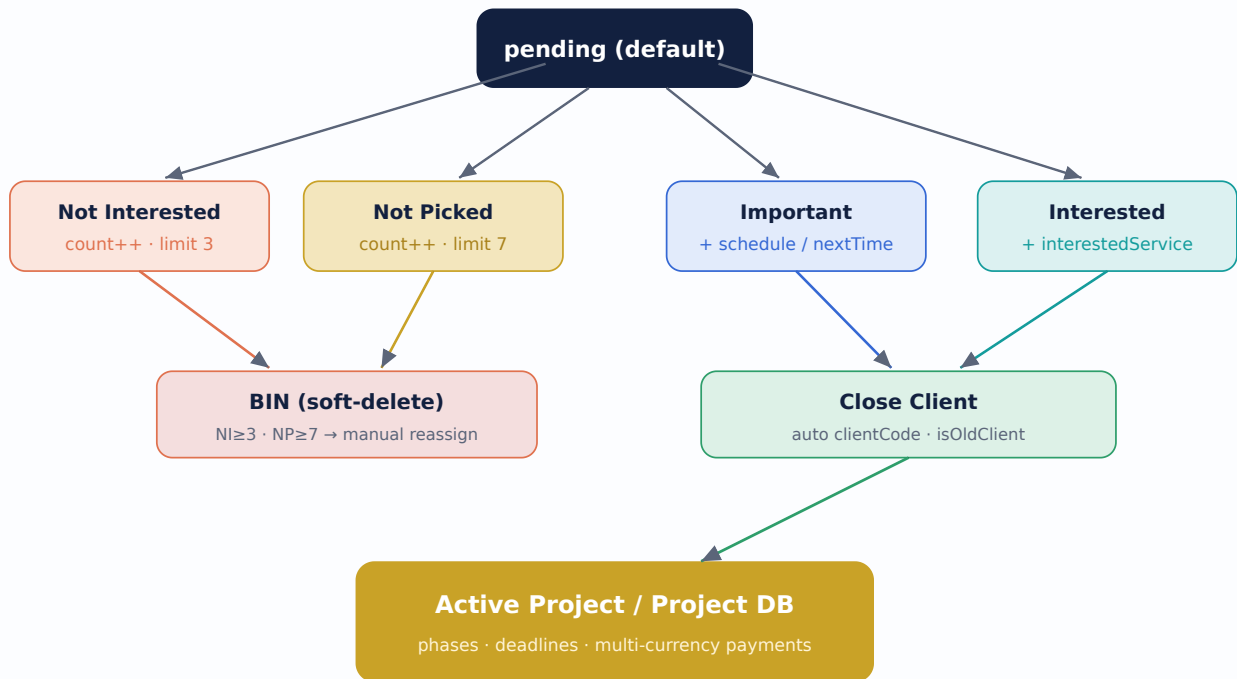


Figure 7 — Lead lifecycle: pending branches to Not Interested / Not Picked (auto-bin), Important (scheduled) and Interested; Close Client promotes to an Active Project.

Automatic Bin / Delete Thresholds

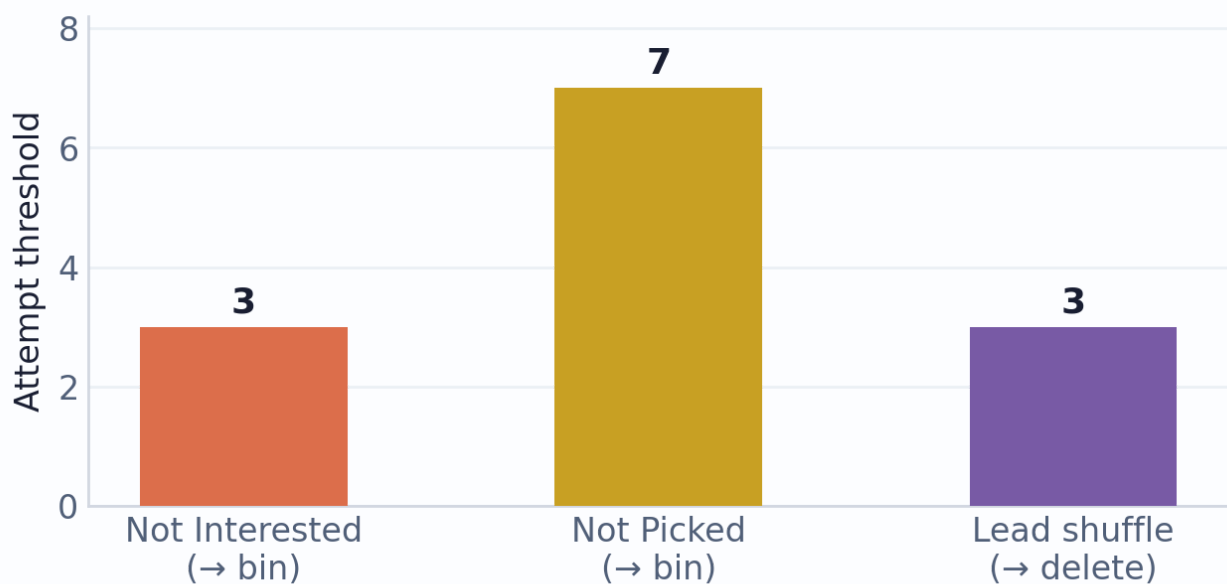


Figure 8 — Automatic thresholds: $NI=3$ and $NP=7$ send a lead to the bin; a lead surviving 3 shuffle attempts is deleted.

CSR Main Dashboard (/csr)

- Uses `/csr/stats` + `/csr/tab-leads` (paginated, DB-level filters)
- Tabs: Today, Pending, Important, Schedule, Interested, Project Won, etc.
- Client-side filters: state, timezone, phone code, upload source
- Status changes open modals (Important, Interested, Project Won, etc.)

Not Interested / Not Picked Limits

- **Not Interested:** `notInterestedCount++` → at **3** → `inBin:true`, `binReason: limit-not-interested`
- **Not picked:** `notPickedCount++` → at **7** → bin with `limit-not-picked`
- Bin leads need manual CSR reassignment

Bin System (utils/binLeads.ts)

A lead is in bin if `inBin === true` OR NP count $\geq$ 7 OR NI count $\geq$ 3. `buildBinWhere(csrId)` powers efficient DB queries (no full-table scan).

Close Client → Project

- Status `Close Client` → auto `clientId` (`{csrCode}-01` ...)
- `isOldClient` from prior won leads with same company
- Appears in Active Projects / Project DB with phases, deadlines, payments
- Each payment stores `amount`, `type`, `link`, `isPaid`, `paidAt`, `createdAt`, `paymentChannel` (AH|SN|CHQ|HM), `currency` (USD|CAD|PKR), `accountsMonth`

Notes behaviour

- **Add** → new `createdAt` → latest on top
- **Edit** (CSR + Admin/Manager PUT) → `createdAt` reset to now → moves to top
- Accounts dated notes: editing bumps `date` to today and re-sorts

Pipeline heal & shuffle

- `healPipelineLeaksForCsr` dedupes won/pending conflicts — throttled once per CSR per 2 min
- **Lead Shuffle (cron):** midnight, then every 2 days; reassigns `not completed` round-robin; max 3 attempts → deleted + report to Cloudfinary

12 Uploads & Data Import

Admin Upload Flow (/admin/uploads)

1. Wizard: category, reset mode, company, source (optional), CSR selection, preview
2. `POST /api/uploads` — Excel parsed, duplicates by phone/email
3. Leads distributed round-robin to selected CSRs
4. File archived to Cloudinary

Upload Categories

`call-data` `new-client` `old-client` `interested` `important`
`active-projects`

Upload Source

- `name` — admin-only label
- `sudoName` — shown to CSRs as lead source

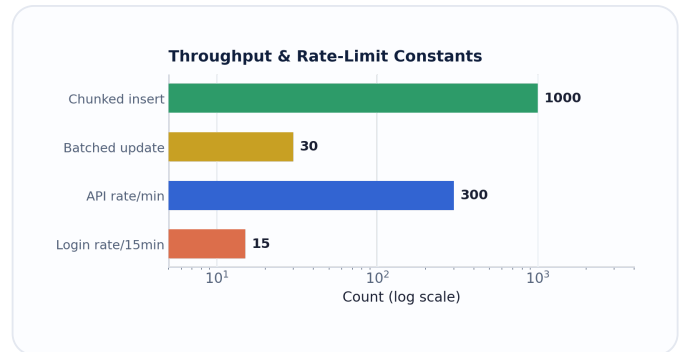


Figure 9 — Bulk-import & rate-limit constants: inserts chunked at 1000 rows, updates batched at 30, API 300/min, login 15/15min (log scale).

13 Chat / Messages System

Messaging is **WebSocket-first** with a REST fallback, gated by a per-user master switch and org-pool access control. The same socket also carries presence and login-attempt pushes.

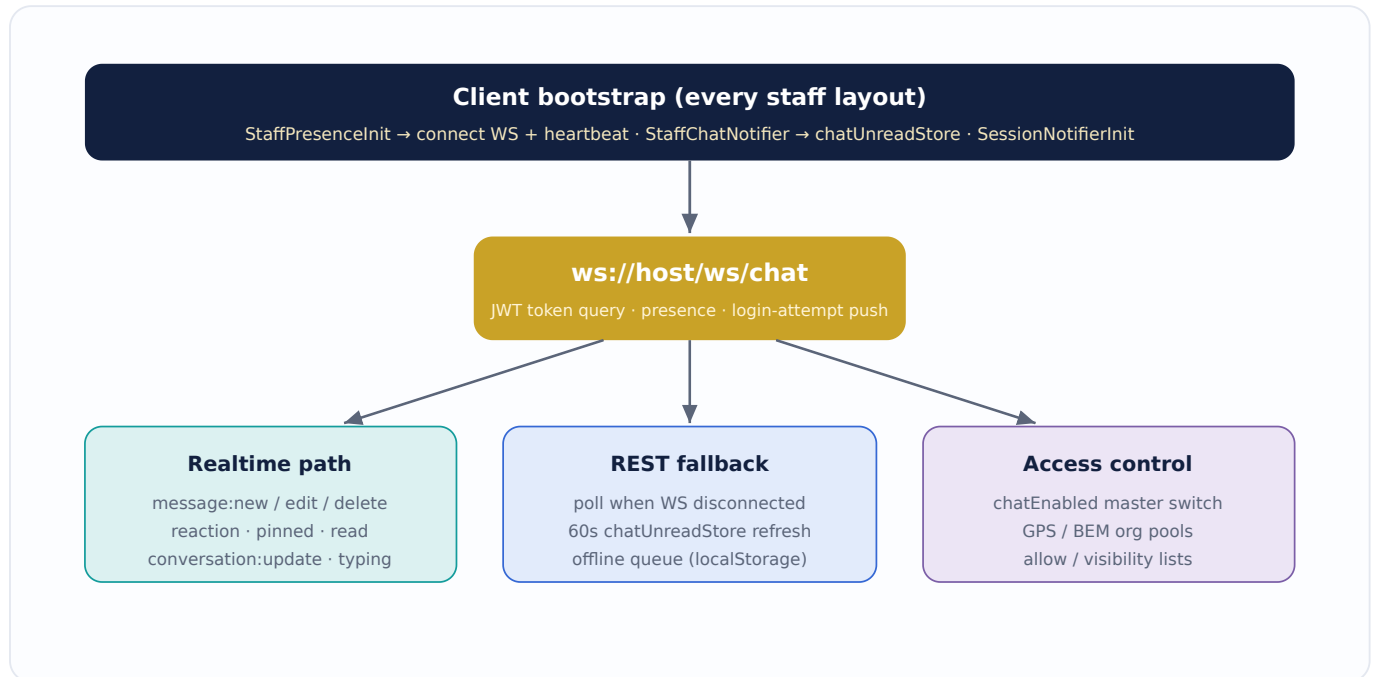


Figure 10 — Chat architecture: a single /ws/chat socket drives realtime events, with REST polling fallback and an org-pool access-control layer.

Chat Access — Org Pools

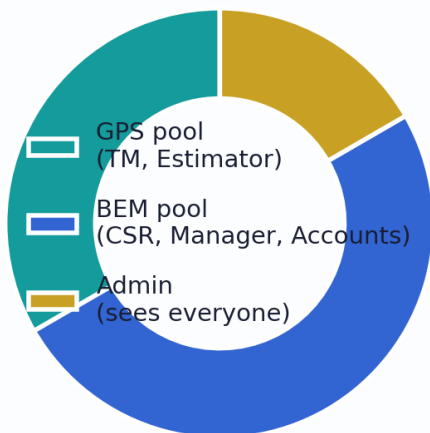


Figure 11 — Org pools: GPS (TM, Estimator) and BEM (CSR, Manager, Accounts); Accounts bridges both; Admin sees everyone.

Access control (chatAccess.ts)

Rule	Detail
chatEnabled	Master off → no directory, no send, WS rejected
Org pools	GPS: TM, estimator · BEM: csr, manager, accounts · Admin: everyone
Visibility	GPS↔GPS+admin+accounts · Accounts bridges both · BEM↔BEM+admin
chatAllowedUserIds	Empty = unrestricted within org; else listed IDs only
chatVisibleToUserIds	Empty = appears in directories; else only listed viewers
Admin UI	Users page toggles ACL fields on PUT /users/id

WebSocket Events (chat/chatSocket.ts)

Client: ping , presence , watch , typing

Server: message:new , message:edit , message:delete , message:reaction , message:pinned , conversation:update , read , presence , typing , auth:login_attempt (session alert — not a chat message)

Features

- Direct + group conversations
- Text, image, file, voice (Cloudinary, up to 200 MB)
- Reactions, replies, mentions, pin, per-user archive
- Read receipts via `lastReadAt`
- Offline queue (`chatOfflineQueue.ts` , `localStorage`)
- Sound + desktop Notification API (chat only)
- **Admin stealth:** read any viewable user's threads (read-only)

Entry points

Portal	Chat UI
Admin	AdminChatPanel (stealth + own)
Manager	ChatPanel (no stealth)
CSR	TopNavbar ChatPanel or <code>/csr/chat</code>
Technical / Estimator	Sidebar ChatPanel (GPS pool)
Accounts	Sidebar ChatPanel (bridge)

14 Notifications & Alerts

NOT THE SAME AS MESSAGES

Bells/inboxes are separate from the MessageSquare unread badge. Desktop OS notifications apply to **chat** only (except the CSR bell's "Enable message notifications" CTA, which grants chat permission).

Portal matrix

Portal	Bell / inbox	Shows	Transport	Dismiss
Admin	Bell	Meetings + latest technical notes	Poll 25s (cache 20s)	localStorage; Clear-all
Manager	Bell	Meetings only	On mount / open (cache 60s)	Per-item localStorage
Technical	Bell	TechnicalNotification inbox	Poll 20s when visible	Server readAt / DELETE
CSR	Bell (TopNavbar)	Own calls + 24h CallLog stats	Cache 60s, no poll	localStorage
CSR	Dashboard banner	scheduledCount > 0 amber banner	From stats	React state
Estimator / Accounts	—	No product bell	—	—
All portals	Toast (not bell)	Concurrent login attempt	Poll 10s + WS	POST ack

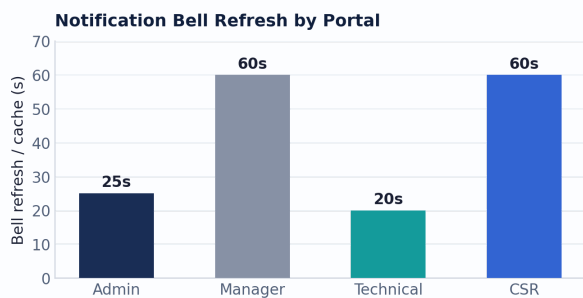


Figure 12 — Bell refresh cadence per portal: Admin 25s (active poll), Technical 20s, Manager & CSR fetch on demand (cache 60s).

TM inbox — TechnicalNotification

Trigger	Source	Title
Project field update (diff)	admin	Project updated
Request-revision	admin	Revision requested
Takeoff complete	estimator	Takeoff completed
Technical note	estimator	Estimator note

Watched fields: phase/status, work status, deadlines (incl. Morning/COB), title/code/scope, bid instruction, service, estimator assignments.

Shared dismiss store & session alerts

- **notificationDismiss.ts** → localStorage key `crm_dismissed_notifications` (JSON id array); used by Admin/Manager/CSR meeting & tech-note IDs — **not** TM inbox (DB) or session notifier
- **Admin/Manager payload A** — `scheduledCalls` : important leads with `nextSchedule ≥ today` and `nextTime` (max 30)
- **Payload B** — `technicalNotes` : latest note JSON; id `tn- $\{leadId\}$ - $\{noteId\}$` → /admin/active-projects
- **Session alerts:** `SessionNotifierInit` in every layout · `useSessionNotifier.ts` (pauses when tab hidden) · `LoginAttemptAlert` via `sessions.ts` · WS `notifyLoginAttempt` · email via `securityEmail.ts`

Messages vs Notifications

	Notifications (bell)	Messages (chat)
Control	Bell	MessageSquare
Badge	Meetings / notes / TM unread	Sum of conversation unreadCount
Transport	HTTP poll / fetch	WebSocket + 60s fallback
Desktop OS notifs	No	Yes (if permission)

15 Presence, Heartbeat & Work Sessions

Two parallel channels

Channel	Purpose
User online	Admin "online" charts / isOnline — /heartbeat, /offline, cron
Chat presence	Green dots in Messages — /chat/status or WS presence

Auto-offline cron (60s): sets `isOnline:false` if `lastActive` > 90s; circuit breaker on DB errors. **Does not close work sessions.**

Work sessions (CsrWorkSession)

Shared by CSR and Estimator (`csrSessionTracking.ts`).

Field	Meaning
loginAt / logoutAt	Check-in / check-out
dashboardMs / awayMs	Tab visible vs hidden
lastHeartbeatAt / lastVisible	Heartbeat bookkeeping
deviceId / browser / ip	Device identity
closedReason	Manual / 16h auto-close / etc.

Polling & Heartbeat Cadence

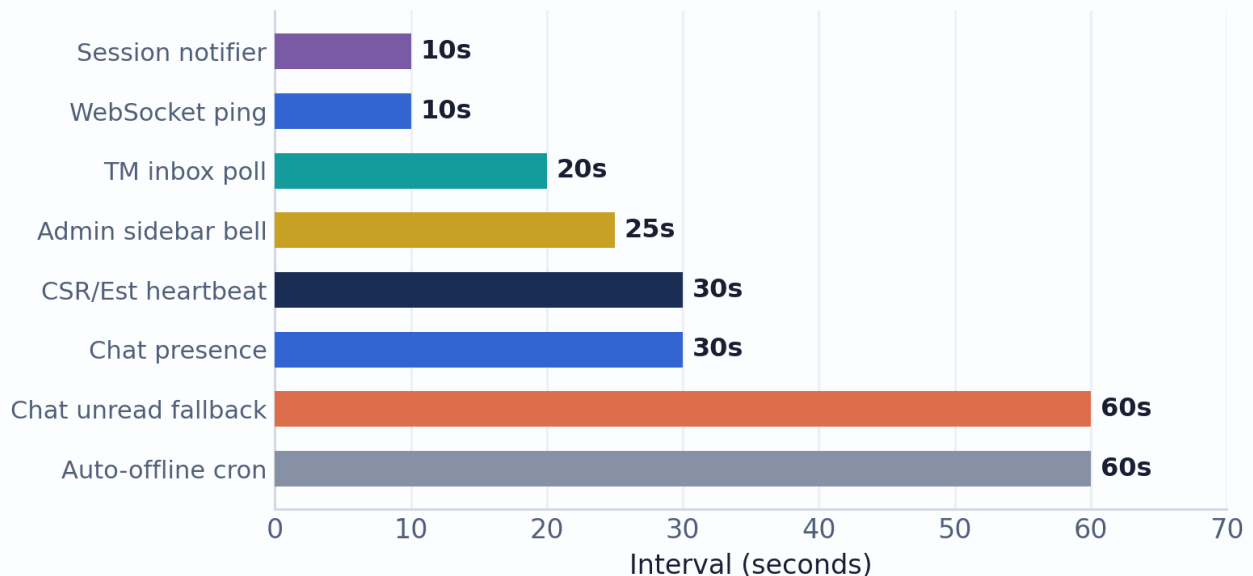


Figure 13 — The complete polling / heartbeat cadence, from the 10s session-notifier and WS ping up to the 60s chat fallback and auto-offline sweep.

`visibleInterval.ts` pauses timers when the browser tab is hidden.

16 Background Jobs (Cron)

Background Cron Jobs



Figure 14 — Four scheduled jobs run inside the API process: the 60s offline sweep, the 3:30 AM daily report, the every-2-days lead shuffle, and the 24h CSR deactivation.

Job	Schedule	Purpose
Auto-offline	Every 60s	<code>isOnline:false</code> if <code>lastActive</code> > 90s; circuit breaker on DB failures; does not close work sessions
Daily report	3:30 AM daily	Yesterday's leads → JSON → Cloudinary → Report
Lead shuffle	Midnight, every 2 days	Reassign not-completed leads (max attempts → delete + Cloudinary report)
CSR deactivation	Every 24h + on startup	Deactivate CSRs inactive 90+ days

Manual shuffle: `POST /api/admin/run-shuffle`

17 Caching & Performance

The backend uses an in-process `Map` with TTL (not Redis — per-process only), invalidated on mutations. TTLs are tuned so hot reads stay fast while writes propagate within seconds.

Backend In-Memory Cache TTLs

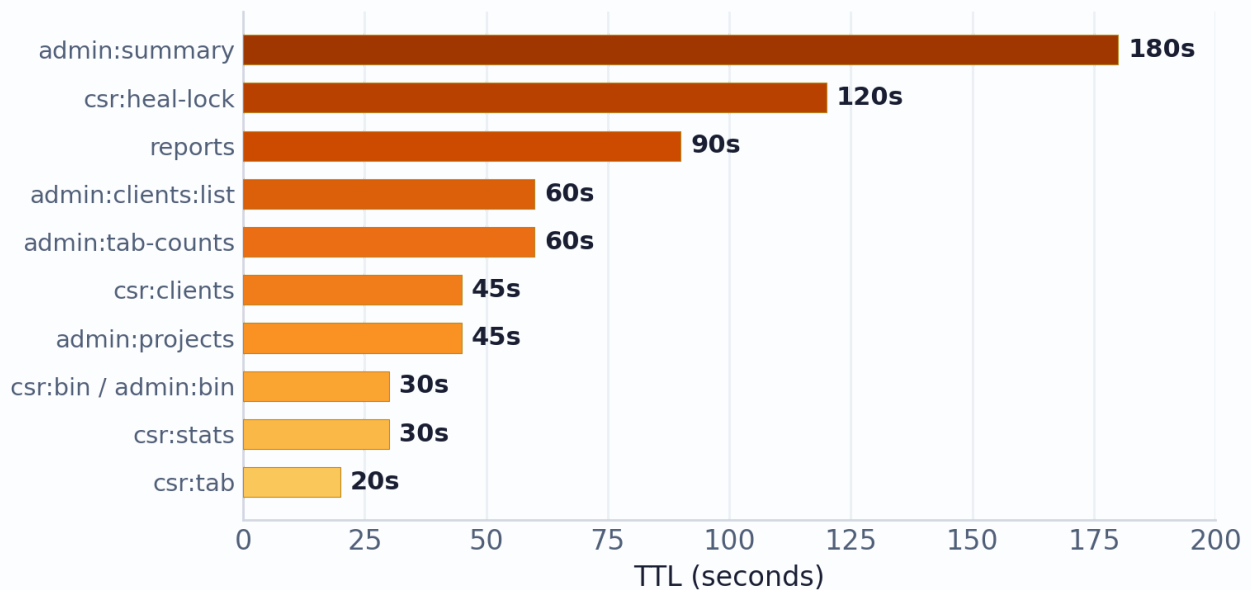


Figure 15 — Backend cache TTLs, from the 20s CSR-tab cache up to the 180s admin summary. Filtered admin requests bypass the shared key.

Backend cache keys

Key pattern	TTL
admin:clients:summary	180s
admin:clients:tab-counts	60s
admin:clients:list:*	60s
admin:bin:*	30s
admin:projects:*	45s
csr:stats:*	30s
csr:tab:*	20s
csr:clients:*	45s
csr:bin:*	30s
csr:heal-lock:*	120s
Report controller	90s / 300s

Performance patterns

1. Lean Prisma selects (max 5 notes on client list)
2. DB pagination on tab-leads & admin clients
3. `buildBinWhere` — bin queries at DB level
4. Cache-first reads (clients, bin, projects skip heal on GET)
5. Throttled heartbeat DB writes ($\leq$ every 30s)
6. Throttled pipeline heal ($\leq$ once/CSR/2 min)
7. Bulk upload batching (1000 insert / 30 update)
8. Optional Prisma rate-limit store (only when flagged)
9. Compound indexes on Lead
10. WebSocket push instead of constant polling

Frontend cache modules

Module	TTL	Endpoints
authMeCache	60s	/auth/me
adminSummaryCache	60–120s	summary, tab-counts (+ sessionStorage)
csrTabApiCache	45s	stats, tab-leads, potential-leads
csrApiCache	28s	clients, leads, bin, projects, NI

reportsCache	300s	/reports
chatApiCache	12-120s	conversations, messages, members

HOSTING NOTE

Run `npx tsc` after src changes — production uses committed `dist/`. Keep heartbeat/cron intervals (60s offline, 30s presence) on \$25 plans. Only set `RATE_LIMIT_STORE=prisma` for multi-instance rate limiting.

18 External Services

Service	Purpose	Config
MongoDB	Primary database	<code>DATABASE_URL</code>
Cloudinary	Uploads, reports, profiles, CNIC, chat files	<code>CLOUDINARY_*</code>
Gmail SMTP	2FA OTP, login alerts	<code>SMTP_USER</code> , <code>SMTP_PASS</code>
Cloudflare Turnstile	Login CAPTCHA	<code>TURNSTILE_SECRET_KEY</code>
Have I Been Pwned	Password breach check	Public API

19 Environment Variables

Backend (.env)

Variable	Req (prod)	Description
PORT	No	Default 5000
DATABASE_URL	Yes	MongoDB connection string
JWT_SECRET	Yes (≥32)	JWT signing
FRONTEND_URL	Yes	CORS + cookies
NODE_ENV	No	<code>production</code> enables strict checks
SMTP_USER / SMTP_PASS	Rec.	2FA email
ADMIN_ALLOWED_IPS	Opt.	Comma-separated admin IPs
TURNSTILE_SECRET_KEY / _SITE_KEY	Opt.	Turnstile verify / captcha-config
RATE_LIMIT_STORE	Opt.	<code>prisma</code> for distributed rate limit
SESSION_DEFAULT_HOURS	Opt.	Default 12
SESSION_REMEMBER_HOURS	Opt.	Default 168 (7 days)
SESSION_STALE_MS	Opt.	Stale session / OTP timing
BUSINESS_TIMEZONE / _DAY_START_HOUR	Opt.	Business calendar
CLOUDINARY_CLOUD_NAME / _API_KEY / _API_SECRET	Yes	Cloudinary

Frontend

Variable	Description
BACKEND_URL	Server-side proxy target (default <code>http://127.0.0.1:5000</code>)
NEXT_PUBLIC_BACKEND_URL	Used by proxy / WS URL derivation
NEXT_PUBLIC_API_URL	Direct API URL (optional)
NEXT_PUBLIC_WS_URL	WebSocket URL (optional)
NEXT_DIST_DIR	Optional custom Next.js dist dir

20 Local Development

```
# Terminal 1 – Backend
cd backend
npm install
# Create .env: DATABASE_URL, JWT_SECRET, FRONTEND_URL, CLOUDINARY_*
npx prisma generate
npx prisma db push
npm run dev           # http://localhost:5000

# Terminal 2 – Frontend
cd frontend
npm install
# .env.local: BACKEND_URL=http://127.0.0.1:5000
npm run dev           # http://localhost:3000
```

Login: `admin@gmail.com` / `manager@gmail.com` / `csr@gmail.com` — password `123456`

PRODUCTION PARITY

After backend src changes, run `cd backend && npx tsc` to refresh the committed `dist/`.

21 Deployment Notes

Component	Typical host	Start command
Frontend	Vercel	<code>next build && next start</code>
Backend	Render (\$25 plan)	<code>npm start</code> → <code>node dist/server.js</code>

Backend build: `prisma generate && prisma db push && tsc && seed`

Checklist

- `FRONTEND_URL` on backend must match deployed frontend origin
- `BACKEND_URL` on frontend must point to backend
- Rebuild `dist/` before pushing backend changes
- Run `npm run prisma db push` when schema changes (network fields, indexes)
- WebSocket must be reachable at `/ws/chat` on same host (or set `NEXT_PUBLIC_WS_URL`)

22 Every Dashboard — Sources & Merges

This is the full inventory: every portal page, where its data is loaded from, and where it is merged / synced into another module. **Route count:** Admin 9 · Manager 7 (re-exports) · CSR 8 · Technical 6 · Estimator 2 · Accounts 10 ≈ **42 pages**.

22.1 Cross-cutting merges (read this first)

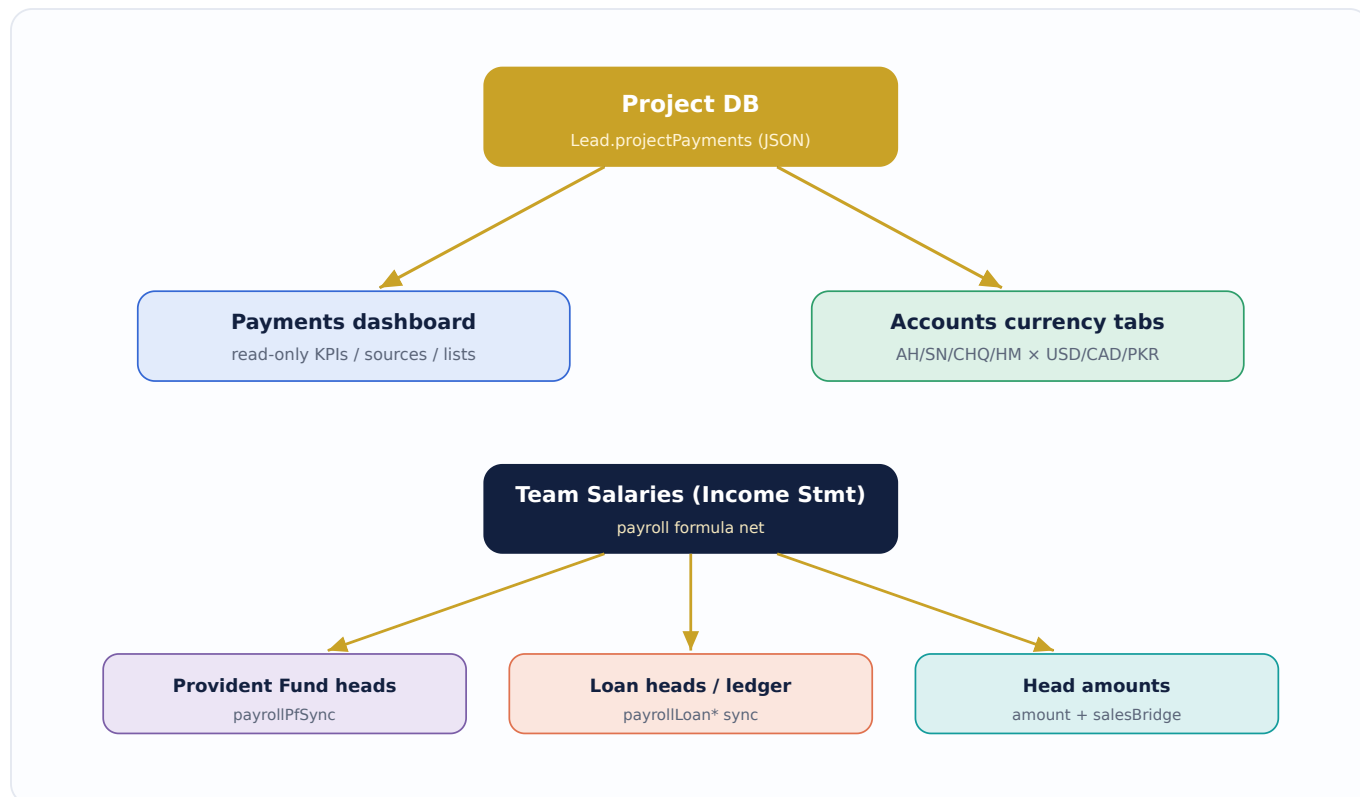


Figure 16 — The two backbone merges: Project DB payments fan out to the Payments dashboard and Accounts currency tabs; Team Salaries drive PF, loan, and head-amount syncs.

Merge	From → To	Trigger	Code
Payments → Accounts currency	projectPayments → Accounts tabs	Project save; Accounts GET	projectCurrencyCollections.ts
Payments dashboard	Same JSON → KPIs / sources / lists	Page load / refresh	paymentsSummary.ts
Payroll PF	Employee PF → PF heads	Income statement recompute	payrollPfSync.ts
Payroll loans	Salary loan ↔ heads ↔ ledger	IS + /payroll-loans*	payrollLoanSync
Payroll amounts	Formula net → sub-head amount	IS load/edit	payrollAmountSync.ts
Sales payroll bridge	Sales/TL JSON → Team Salaries rows	IS normalize	salesPayrollBridge.ts
Currency formula revenue	Currency tabs → IS revenue	IS + monthly reports	currencyFormula.ts
Notes merge	LeadNote + legacy strings	CommentCell / lists	noteHistory.ts
Technical notes	Separate technicalNotes JSON	TM / Admin / Estimator	technicalEstimators.ts
Pipeline heal	Won/pending conflict cleanup	CSR stats/list (throttled)	leadClientPromotion.ts
Won client split	New vs old identity	CSR clients page	splitWonClientsByIdentity

22.3 Payment JSON shape (single source of truth)

Stored on `Lead.projectPayments` . Parsers: backend `projectPayments.ts` , frontend `projectFields.ts` .

Field	Meaning
id	Stable row id
type	Partial / Milestone / Final / Other
amount	Collected amount
link	Slip URL (Admin/Manager)
isPaid	Collected flag
paidAt	Payment date
createdAt	Row history timestamp
paymentChannel	AH · SN · CHQ · HM
currency	USD · CAD · PKR
accountsMonth	Receive month (YYYY-MM)

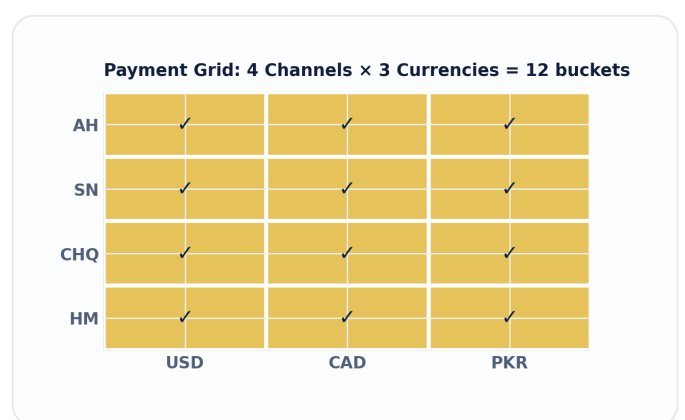


Figure 17 — Every payment lands in one of 12 buckets — 4 collection channels (AH/SN/CHQ/HM) crossed with 3 currencies (USD/CAD/PKR).

22.4 Admin (/admin/*)

Page	Source	Merge / notes
/admin (Dashboard)	GET /admin/clients/summary via adminSummaryCache (+ /csrs, /projects)	KPI tiles, chart series, top performers; stale broadcast after mutations
/admin/leads	GET /admin/clients + tab-counts (same filters); /csrs; lead CRUD	Tab counts follow filters; notes via CommentCell (createdAt bump)
/admin/active-projects	GET /admin/projects; CRUD; excel-template + excel-upload; mark-completed	On save → Accounts currency sync; tabs All/Active/Completed; +N history
/admin/payments	GET /admin/payments-summary?month=	Read-only from projectPayments; AH/SN/CHQ/HM × USD·CAD·PKR
/admin/uploads	upload-sources, trades, CSRs, POST /uploads, history	Interested-by-phone consolidate on upload
/admin/reports	GET /reports; CSR history; session report + edit	reportsCache; session reconcile
/admin/bin	GET /admin/bin; reassign	Same binLeads rules as CSR
/admin/users (admin only)	/users CRUD; upload-pic/cnic; access-history; reset	CNIC image/pdf; legacy raw → Docs viewer
/admin/settings (admin only)	/auth/me, profile, password, 2FA, audit-logs, logout-all	Profile + security audit

FINAL → COMPLETED FLOW

When status is **Final Submission**, Admin sees **Complete** → sets `projectWorkStatus / status` completed, `hiddenOnTechnical:true` (drops from TM), and lands in the **Completed** tab.

22.5 Manager (/manager/*) — re-exports Admin

Item	Detail
Pages	Same as Admin except no Users / Settings
Paths	<code>useStaffBase()</code> → /manager; proxy blocks /admin/* for manager
Chat	Regular ChatPanel — no Admin stealth
Notifications	Meetings bell only
Admin-only APIs	User CRUD, access-history, CSR purge/reset, shuffle run, migrate-notes, report-uploads

22.6 CSR (/csr/*)

Page	Source	Merge
/csr (Call Data main)	/csr/stats, /csr/tab-leads; PUT lead; notes	Pipeline heal on stats; notes merge; cache invalidate
/csr/potential-clients	/csr/potential-leads, stats; update / bin	potentialClientBuckets
/csr/clients	/csr/clients; lead/project create; notes; phone lookup	Identity split; payments JSON (Accounts sync on Admin save)
/csr/active-projects	/csr/projects; update; notes; create	ProjectWorkbenchModal, CommentCell

/csr/not-interested	/csr/not-interested; status/assign/notes	Limits NI=3 / NP=7 → bin
/csr/bin	/csr/bin; restore inBin:false	Reason categories, charts
/csr/chat	/chat/* REST + WS /ws/chat	Conversation merge; unread store; ACL
/csr/settings	/auth/me, toggle 2FA	—

22.7 Technical (/technical/*)

Page	Source / shows
/technical (Dashboard)	GET /admin/projects?technicalView=1&excludeRevisions=1; project-mix pie + workflow-stage bar; top performers
/technical/active-projects	Admin projects (technicalView); assign/takeoff/pricing/final/return/QA; technicalNotes; hide. Hide rules: hiddenOnTechnical OR completed
/technical/revisions	Same page, variant="revisions" / revisions-only filter
/technical/kpi	KPI records CRUD; KPI auto; monitoring — pipeline health, scoreboard
/technical/team	monitoring + ratings — per-estimator workload, timers, hours, overdue
/technical/reports	monitoring + KPI auto + revision projects — monthly charts, stars, Word export

22.8 Estimator (/estimator/*)

Page	Source / shows
/estimator	GET /estimator/dashboard; assigned/pending/takeoff-done/overdue; assignment-mix pie + workload bar; stars
/estimator/my-projects	GET /estimator/projects; notes; timer; complete — timers → TM monitoring

22.9 Accounts (/accounts/*)

Page	Source
/accounts (Currency dashboard)	dashboard; summary; FX — live overlay of Project DB payments
/accounts/income-statement	income_statement + dashboard date + budget + loans — PF, loan, amount, sales bridge
/accounts/balance-sheet	balance_sheet — assets / equity / liabilities
/accounts/cash-flow-statement	cash_flow — bank debit/credit, net
/accounts/total-assets	total_assets — register + proof uploads
/accounts/reports	monthly generate/save — currency formula revenue

Legacy redirects → income-statement: csr-payroll, sales-team-payroll, sales-team-lead-payroll, technical-team-payroll

22.10 Data-flow cheat sheet

User action	Lands in	Also updates
CSR Close Client	Project DB / CSR projects	clientCode, isOldClient
Admin saves payment (channel+currency+month)	Lead.projectPayments	Accounts currency tabs; Payments dashboard next load
Add 2nd/3rd payment	Same JSON array	Project DB +N history; Payments expand
Edit Lead note	LeadNote.createdAt now	Latest note on top everywhere
Edit salary PF / occasional	Income statement day JSON	Head total + PF heads
TM assigns estimator	technicalAssignments	Estimator my-projects + TM monitoring
TM Final Submission	projectPhase: qa_phase	Admin Final Submission + Complete button
Admin Mark Completed	projectWorkStatus + hiddenOnTechnical	TM list removes; Admin Completed tab
Project DB Excel upload	New leads (any date)	Accounts currency sync for paid months
Estimator starts timer	Assignment timer fields	Technical team / KPI monitoring
Estimator takeoff / note	TechnicalNotification row	TM sidebar inbox
Admin revision request	needsRevision + TM inbox	TM Revisions tab
Concurrent login blocked	LoginAttemptAlert + WS	Session toast on all portals

23 Quick Reference

Concern	File
DB schema	backend/prisma/schema.prisma
Server + cron	backend/src/server.ts
Express app	backend/src/app.ts
WebSocket chat	backend/src/chat/chatSocket.ts
Chat ACL (org pools)	backend/src/utis/chatAccess.ts
TM inbox notifications	utis/technicalNotifications.ts
Notification dismiss (local)	frontend/lib/notificationDismiss.ts
Session login-attempt toast	lib/useSessionNotifier.ts
Work sessions	utis/csrSessionTracking.ts
Auth middleware	middleware/auth.middleware.ts
Network access	utis/networkAccess.ts
Bin logic	utis/binLeads.ts
CSR tab queries	utis/csrDashboardQueries.ts
Admin filters / tab counts	utis/adminClientQueries.ts
Payments summary API	utis/paymentsSummary.ts
Project payment JSON	utis/projectPayments.ts · lib/projectFields.ts
Project DB Excel	utis/projectDbExcel.ts
Upload parser	controllers/upload.controller.ts

Concern	File
Portal picker	frontend/app/page.tsx
Login form	components/auth/LoginPortalForm.tsx
API client	frontend/lib/api.ts
Device ID header	frontend/lib/deviceIdentity.ts
Route guard	frontend/proxy.ts
Backend proxy	app/api/backend/[...path]/route.ts
CSR main page	frontend/app/csr/page.tsx
CSR tab hooks	app/csr/hooks/useCsrTabPage.ts
Notes UI	csr/components/shared/CommentCell.tsx
Admin users + CNIC	app/admin/users/page.tsx
Admin leads (filtered counts)	app/admin/leads/page.tsx
Admin / Manager payments	app/admin/payments/page.tsx
Project DB workbench	components/projects/ProjectWorkbenchModal.tsx
Accounts payroll	accounts/utis/payrollFormula.ts
Technical sidebar	technical/components/layout/Sidebar.tsx
Chat panel	components/chat/ChatPanel.tsx
Admin stealth chat	admin/components/layout/AdminChatPanel.tsx
Lead statuses	csr/constants/leadStatuses.ts

Lead Statuses

Status	Display	Notes
pending	Pending	Default
not picked / not completed	Not Picked	Limit: 7
Not Interested / completed	Not Interested	Limit: 3
no owner available	No Owner	Misc tab
not in service	Not In Service	Misc tab
in house	In House	Misc tab
important	Important	+ schedule fields
interested	Interested	+ interestedService
Close Client	Close Client	Becomes active project

END OF DOCUMENTATION

This reference captures the complete BEM CRM codebase as of August 2026 — six portals, ~42 dashboards, real-time chat, multi-currency payments, and the full merge graph — in a single visual document.